

CSCE 620 COMPUTATIONAL GEOMETRY · MODULE 10 · LECTURE NOTES

Range Searching and Proximity

Preprocessing, so the query is cheap.

MODULE OUTCOMES

- Analyse the space–time trade-off in range searching.
- Build range trees and explain fractional cascading.
- Implement nearest-neighbour search with pruning.
- Explain approximate nearest neighbour and when to use it.
- Explain the curse of dimensionality concretely.

1 The space–time trade-off

Structure	Space	Query (2D orthogonal)	Build
Linear scan	$O(n)$	$O(n)$	$O(1)$. Never dismiss it — perfect locality, no branching, and it wins at small n and in high dimensions (§3).
k-d tree	$O(n)$	$O(\sqrt{n} + m)$	$O(n \log n)$. The practical default despite the worse bound, because the space is linear and the layout is cache-friendly.
Range tree	$O(n \log n)$	$O(\log^2 n + m)$	$O(n \log n)$. Buys query time with space — the clearest example of the trade in the subject.
Range tree + fractional cascading	$O(n \log n)$	$O(\log n + m)$	$O(n \log n)$. One log factor removed, free.
Full precomputation	$O(n^2)$ or worse	$O(1 + m)$	Expensive. Included to mark the end of the spectrum.

m is the number of points reported, and no structure can improve on the $O(m)$ term — the answer has to be listed. Bounds of this shape are called *output-sensitive* (Module 03 §2), and the right way to read the table is that the structures differ only in the additive search term.

A RANGE TREE IS A TREE OF TREES

Build a balanced binary search tree on the x-coordinate. Each node of this primary tree corresponds to a contiguous range of x values — the points in its subtree.

At every node of the primary tree, store a secondary balanced tree on the y-coordinate, containing exactly the points in that node's subtree.

A query x-interval decomposes into $O(\log n)$ canonical nodes whose x-ranges exactly tile the interval and are pairwise disjoint. Search each of their secondary y-trees for the query's y-interval and report what is found.

Total $O(\log^2 n + m)$. The space is $O(n \log n)$ rather than $O(n)$ **because each point appears in the secondary tree of every ancestor** — $O(\log n)$ of them. That extra space is exactly what was bought, and it generalises: a d-dimensional range tree costs $O(n \log^{d-1} n)$ space for $O(\log^d n + m)$ query.

FRACTIONAL CASCADING REMOVES ONE LOGARITHM

The $O(\log^2 n)$ arises from performing a fresh binary search in each of $O(\log n)$ secondary trees — and crucially, every one of those searches is for the *same* y value, in lists that are closely related (a parent's list is the merge of its children's).

So link them. Store with each element of a node's sorted list a pointer to the position of the corresponding value in each child's list — that is, where that value would fall.

Then only the first search costs $O(\log n)$; every subsequent one is $O(1)$, obtained by following the stored pointer and adjusting by at most one position.

The result is $O(\log n + m)$. The technique generalises far beyond range trees: whenever an algorithm repeats a search for the same key across a family of related sorted lists, fractional cascading applies. It is one of the genuinely reusable ideas in this course, and it is worth recognising the pattern rather than only the application.

2 Nearest neighbour

```
NEAREST(node, q, best):
    if leaf: return min over points in node

    # Descend the NEARER child first. This is the whole trick.
    near, far = children ordered by which side q lies on
    best = NEAREST(near, q, best)

    # Visit the far side only if it could hold something closer.
    if dist_to_split_plane(q, node) < best:
        best = NEAREST(far, q, best)
    return best
```

For k nearest: keep a bounded max-heap of size k and prune against its largest element.

Descending the nearer child first is not an optimisation detail — it is what makes the pruning work at all. Visiting the near side first makes best small early, which makes the far-side test succeed far more often. Reversing the order yields a correct implementation that is dramatically slower, and this is the single most common performance bug in hand-written nearest-neighbour code.

3 High dimensions

THE CURSE OF DIMENSIONALITY, CONCRETELY

Distances concentrate. As the dimension grows, the ratio between the distance to the farthest point and the distance to the nearest point approaches 1 for most natural distributions. **So 'nearest neighbour' gradually stops being a meaningful notion** — everything is roughly equidistant, and which point is nearest is determined by noise.

Volume moves to the corners. The ratio of the volume of the inscribed sphere to that of the unit cube goes to zero rapidly with dimension. **So a ball-versus-box pruning test — which is exactly the test in the code above — almost never prunes**, because the query ball overlaps essentially every box.

Almost everything is on the boundary. Nearly all the volume of a high-dimensional ball lies in a thin shell just inside its surface, so there is no meaningful 'interior' to recurse away from.

Together these mean exact nearest-neighbour search above roughly twenty dimensions costs more than a linear scan, which has perfect memory locality, no branch misprediction, and vectorises cleanly (CSCE 735 Module 02). **The tree is not merely no better; it is actively worse**, which is a result worth internalising before reaching for a spatial structure on embeddings.

Method	Idea	Character
Locality-sensitive hashing (LSH)	Hash with a family designed so that near points collide with high probability and distant ones rarely do.	Provable approximation guarantees, and the trade between recall and cost is explicitly tunable. The theoretically cleanest option.
HNSW	A navigable small-world graph with long-range links in upper layers; greedy descent from a coarse layer to a fine one.	The practical default now — excellent recall-versus-speed curves and the basis of most vector databases.
Product quantisation	Split the vector into subvectors, quantise each against a small codebook, and compute distances on the codes.	Very large memory savings — often 10 to 50× — which is what makes billion-scale search feasible at all.
Random projection trees	Split on random directions rather than on axes.	Simple, and a good baseline. Avoids the axis-alignment weakness of k-d trees.
Inverted file (IVF)	Cluster the dataset, then search only the nearest few clusters.	Scales well and composes with product quantisation, which is the standard industrial combination.

Approximation is not a compromise in this setting. When distances concentrate, the exact nearest neighbour is only marginally more meaningful than a good approximate one — the difference between them is frequently smaller than the noise in how the vectors were produced. **Insisting on exactness here is paying a large cost for a distinction the data does not support**, which is a useful reframing.

4 In practice

- **Broad-phase collision detection:** a uniform grid or sweep-and-prune, not a tree. **Every object moves every frame**, so $O(1)$ update cost dominates the query bound entirely (Module 07 §4).
- **Audio sources and AI perception radii:** a spatial hash, rebuilt or incrementally updated each frame. The queries are small-radius and numerous.
- **Nearest navmesh polygon:** a k-d tree or BVH over the navmesh, which is static and so can afford a good structure (Module 11).
- **Photon lookup and particle neighbour finding:** a k-d tree built once per frame, which is the classic photon-mapping arrangement (CSCE 647) and the SPH neighbour search (CSCE 649 Module 12).
- **And dynamic objects break every static structure**, which is why grids and hashes dominate engine code despite having worse asymptotic bounds than the trees in this module. **The query bound is rarely what decides it** — update cost, memory layout, and cache behaviour usually do, which is CSCE 735's lesson applied to data structures.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
de Berg et al. — Computational Geometry, chapters 5 and 10 (range searching)	Range trees and fractional cascading, with the analyses of §1.
Mount — CMSC 754, range searching lectures (free PDF)	Clearer on the canonical-node decomposition than most treatments.
Malkov & Yashunin — Efficient and robust approximate nearest neighbor search using HNSW (free)	The §3 default method , from its authors.
Facebook Research — FAISS (free)	Production implementations of IVF, product quantisation, and HNSW, with benchmarks. The practical companion to §3.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement 2D orthogonal range search by linear scan, k-d tree, and range tree. Measure all three against n .
- 2 **Confirm the $\sqrt{n}$ and $\log^2 n$ query behaviours empirically** and compare against the predicted curves.
- 3 Measure the actual memory of the range tree and confirm $O(n \log n)$.
- 4 **Implement fractional cascading** and measure the improvement.

- 5 Implement nearest neighbour with pruning. **Then reverse the child order and measure the slowdown.**
- 6 Extend to k nearest with a bounded max-heap.
- 7 **Measure the distance concentration ratio** — farthest over nearest — as dimension goes from 2 to 200. Plot it.
- 8 Measure the inscribed-sphere volume ratio against dimension.
- 9 Find the crossover dimension where a linear scan beats your k -d tree.
- 10 Use FAISS or an HNSW implementation and plot recall against query time.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Give the space and query bounds for four range-searching structures.
- 2 Why can no structure beat the $O(m)$ term?
- 3 Describe a range tree and explain why its space is $O(n \log n)$.
- 4 What is fractional cascading and what does it generalise to?
- 5 Why must nearest-neighbour search descend the nearer child first?
- 6 Give three concrete statements of the curse of dimensionality.
- 7 Why does a linear scan win in high dimensions?
- 8 Name five approximate methods and why approximation is appropriate.
- 9 Why do engines use grids despite worse bounds?